

Architecture & Build Process

This document is for humans reading the codebase. It covers two things:

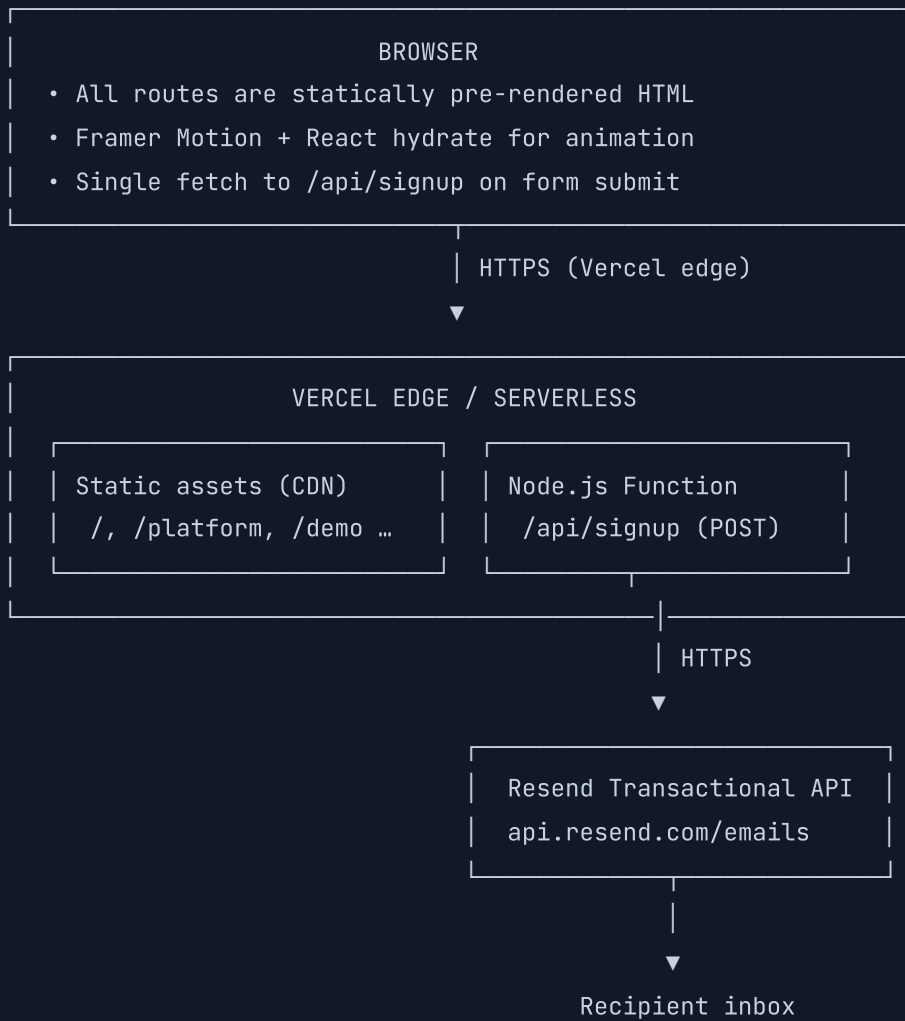
- **Part I — Architecture:** how the system is structured today
- **Part II — Build process:** the chronological steps used to get it here
- **Part III — Decisions:** why each major choice was made (and what was rejected)
- **Part IV — Reference:** annotated file map and diagrams

For state-of-the-project handoff (open TODOs, current version, what to not do), see `CLAUDE.md` instead.

Part I — Architecture

1. System overview

Adaptive Antivirus is a **single-codebase Next.js 15 application** that serves a marketing website, four deep-link product pages, an interactive demo, and a single backend route for email signup.

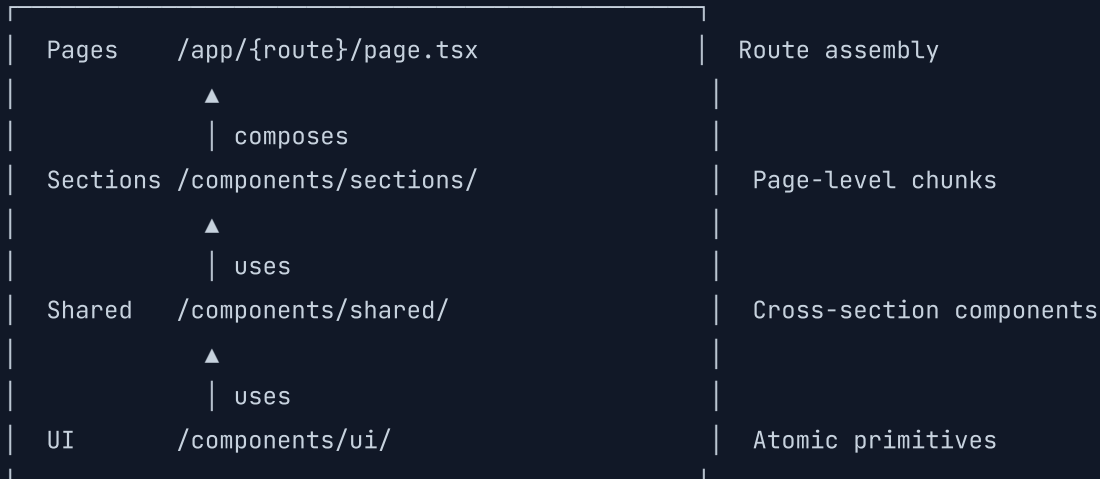


2. Tech stack (and what each piece does)

LAYER	CHOICE	JOB
Runtime	Node.js 22.11 LTS	App server + build toolchain
Framework	Next.js 15.5 (App Router)	Routing, RSC, static generation, API routes
UI library	React 19	Component model + hooks
Language	TypeScript 5.7 (strict)	Type safety, no <code>any</code>
Styling	Tailwind CSS v4 beta	Utility classes + <code>@theme</code> design tokens
Animation	Framer Motion 11	Viewport-triggered fades, the demo state machine, the cycle SVG
Icons	Lucide React	Single icon set across the entire UI
Charts	Recharts	The single attack-growth area chart on <code>/</code>
Theme	next-themes (forced dark)	Locks the brand to dark mode
Email	Resend SDK	Transactional welcome email
Hosting	Vercel	Edge CDN + serverless functions + builds
Source control	GitHub (public repo)	Code history + collaboration

3. Frontend architecture

Component layers (bottom-up)



- **UI primitives** (9 files): Button, Card, Pill, ProgressBar, StatCard, GlowCard, CodeBlock, NumberBadge, SectionHeader. These are the smallest unit — no business logic, just visual atoms with variants.
- **Shared components** (8 files): Navbar, Logo, GridBackground, ParticleField, SystemActiveBadge, SectionShell, ThreatMap, CheckRow. Reused across multiple sections.
- **Sections** (~28 files): Each maps to a distinct part of a page. Eight of them are route-unique (4 heroes + 4 differentiating sections); the other 20 form the home-page scroll.
- **Pages** (`app/*/page.tsx`): Each route is a thin file that imports sections in order. No data fetching, no state — just composition.

State management

There is **no global state library** (no Redux, Zustand, Jotai, Context, etc.). Reasons:

- The site is mostly static content
- The few interactive pieces (HowItWorks tabs, demo state machine, signup form, FAQ accordion) each own their own state via `useState`
- Route data is passed as static imports from `lib/constants.ts`

Design token system

A single source of truth in `app/globals.css` :

```
@theme {
  --color-background: #0a0e1a;
  --color-primary: #7cc4ff;
  --color-text-primary: #e8eef7;
  /* ...etc */
}
```

Custom utilities defined alongside (`.bg-grid` , `.gradient-border` , `.scanline` , `.glass` , `.text-gradient-cyan`) so any component can apply branded effects with one class.

Animation system

Centralized in `lib/motion.ts` :

```
export const fadeUp: Variants = { ... };
export const fadeUpStagger: Variants = { ... };
export const fadeUpItem: Variants = { ... };
export const viewportOnce = { once: true, amount: 0.2 };
```

Every section uses these presets so animations feel coherent across the site.

`useReducedMotion` and a global CSS rule respect `prefers-reduced-motion` for accessibility.

4. Backend architecture

There is exactly **one** backend endpoint:

```
POST /api/signup
├─ parse JSON body
├─ honeypot bot check (silent success)
├─ email format validation (regex + length cap)
├─ env var sanity check
├─ call Resend SDK
└─ return { ok: boolean, error?: string }
```

Source: `app/api/signup/route.ts` — runs on Node.js runtime (not Edge) because the Resend SDK uses Node-specific APIs. Single file, ~70 lines, no database.

Why no database?

Storing signups was not in scope. Resend's dashboard keeps a delivery log, and that log doubles as the early-access list for now. If broadcast emails become a need, the next step is a Vercel KV or Supabase table — see `CLAUDE.md` Open TODOs.

Email content

`lib/welcomeEmail.ts` exports:

- `WELCOME_SUBJECT` — single line
- `welcomeEmailHtml()` — table-based dark-mode HTML (~80 words of copy)
- `welcomeEmailText()` — plain-text fallback (sent in same `send()` call)

Both formats sent together — best practice for deliverability and accessibility.

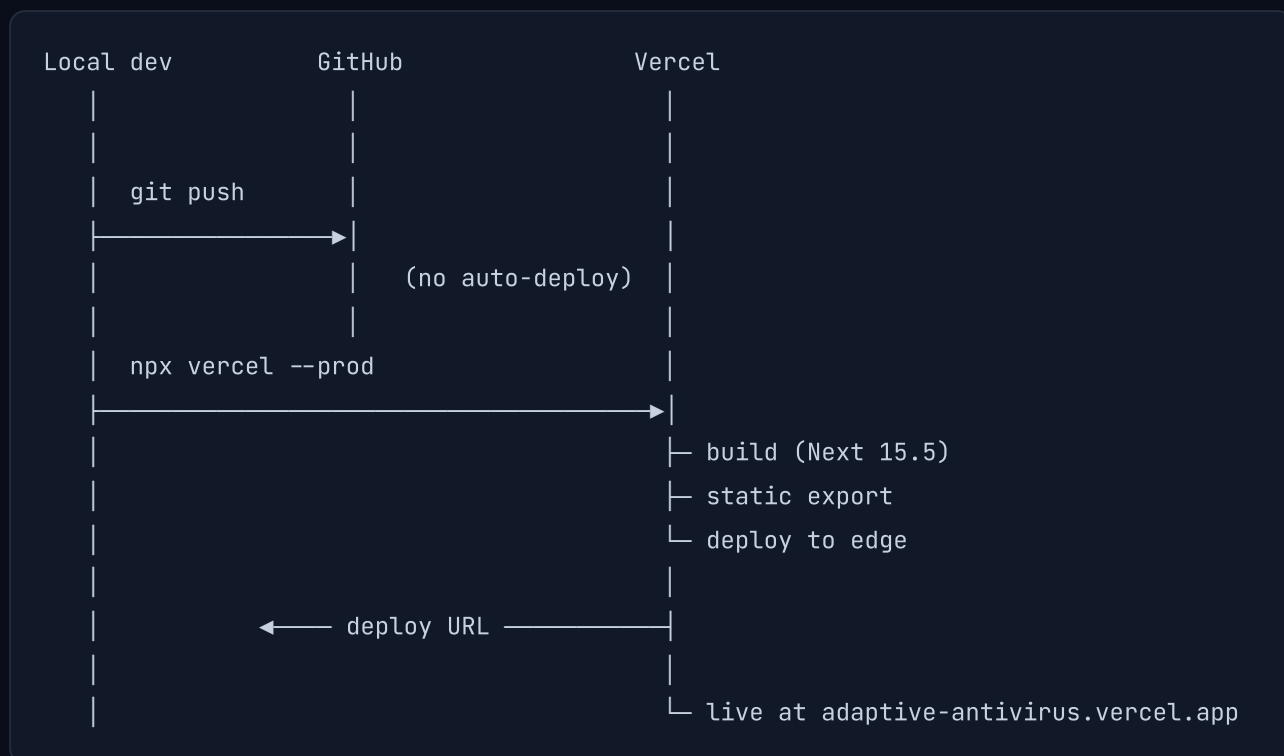
5. Routing & static generation

Six static routes plus one Node function:

PATH	TYPE	PRE-RENDERED?
<code>/</code>	Static page	✓
<code>/platform</code>	Static page	✓
<code>/how-it-works</code>	Static page	✓
<code>/coverage</code>	Static page	✓
<code>/demo</code>	Static page (client interactivity hydrates after)	✓
<code>/pricing</code>	Static page	✓
<code>/api/signup</code>	Node serverless function	✗ (on-demand)

All non-API routes are static at build time — the home page is 110 KB, deep links are 5–13 KB each. First Load JS is dominated by Recharts on the home page (~50 KB).

6. Deployment architecture



Important: GitHub-to-Vercel auto-deploy is **not yet wired**. Pushing to `main` won't trigger a redeploy until the Vercel project is connected to the repo via the dashboard (Settings → Git). Until then, every deploy is a manual `npx vercel --prod`.

Environment variables (Resend) are set both locally (`.env.local`, gitignored) and in Vercel production via `vercel env add`.

Part II — Build process (chronological)

The site was built across two sessions on 2026-05-02 and 2026-05-03 in eight phases. This is the actual order; nothing was skipped or reordered.

Phase 1 — Bootstrap (no Node, no nothing)

The starting machine had no Node.js, no Homebrew, no GitHub CLI, no Vercel CLI installed. Every dev tool was installed user-locally to `~/.local/` to avoid `sudo`.

1. Download Node 22.11.0 LTS tarball (darwin-x64) from nodejs.org
2. Extract to ~/.local/node
3. Symlink node, npm, npx into ~/.local/bin (already on PATH)
4. Verify: node --version → v22.11.0

Phase 2 — Project scaffold

Next.js 15 was scaffolded **manually** rather than via `create-next-app` — for tighter control over every file.

1. `mkdir -p adaptive-antivirus/{app,components/{ui,sections,shared},hooks,lib,public}`
2. Hand-write package.json with pinned versions
3. Hand-write tsconfig.json (strict, ES2022, bundler module resolution)
4. Hand-write next.config.ts, postcss.config.mjs, .gitignore
5. `npm install` (95 packages, ~2 minutes)

Phase 3 — Design system

Built bottom-up: tokens → utilities → primitives → shared components.

1. app/globals.css with @theme block (CSS-variable design tokens)
2. lib/utils.ts (cn helper), lib/motion.ts (Framer Motion presets)
3. hooks/useInViewport, hooks/useCountUp
4. UI primitives: Button, Card, Pill, ProgressBar, StatCard, GlowCard, CodeBlock, NumberBadge, SectionHeader
5. Shared: Logo, Navbar, GridBackground, ParticleField, SystemActiveBadge, SectionShell, CheckRow, ThreatMap

Phase 4 — The 20 home sections

Built in five batches of four sections each. Each section is its own `.tsx` file with section-local state and animation. All copy lives in `lib/constants.ts` — the single source of truth.


```
Batch 1: Hero
Batch 2: CyberReality, LegacyFails, MalwareSpreads, Breakthroughs
Batch 3: HowItWorks, Architecture, Coverage, ConsoleSupport
Batch 4: ThreatMatrix, ResponseTimeline, Privacy, ProductExperience
Batch 5: Competitive, Moat, Pricing, Roadmap
Batch 6: Thesis, CTA, Footer (+ ThreatMapSection bonus)
```

After each batch: type-check, fix errors, verify dev server.

Phase 5 — Deep-link routes (V1, then V2)

V1 (initial): four routes that all reused the home `Hero` component plus a few sections each. Functional but not visually distinct — every route opened with the same headline.

V2 (same session, ~30 minutes later): rewrote each route with its own hero and one route-unique section.

Route	Unique hero	Unique section
<code>/platform</code>	<code>PlatformHero</code>	<code>EngineSpec</code>
<code>/how-it-works</code>	<code>HowHero</code>	<code>CycleDiagram</code>
<code>/coverage</code>	<code>CoverageHero</code>	<code>OSWall</code>
<code>/pricing</code>	<code>PricingHero</code>	<code>PricingFAQ</code>

Phase 6 — GitHub publish

1. Install gh CLI 2.65 user-locally (same tarball pattern as Node)
2. `git init -b main + set local user.name / user.email`
3. Initial commit (67 files)
4. User runs ``gh auth login`` (browser OAuth — only step requiring user input)
5. `gh repo create --public --source=. --push`
6. CLAUDE.md handoff doc written and pushed as second commit

Phase 7 — Interactive demo

The `/demo` route was added later in the second session — a 7-phase client-side state machine simulating the full adaptive cycle.

States: idle → detect → analyze → adapt → harden → share → complete

Components:

- InteractiveDemo: state machine + chained setTimeouts
- DemoHero: amber "Simulation only" disclaimer
- DemoExplainer: post-demo cards explaining each phase
- Fake threat name: TROJAN.SIM.0XF7A2 (the ".SIM." infix is intentional)

Phase 8 — Production deploy on Vercel

1. `npx next build` (verify all 9 pages prerender clean)
2. User runs ``npx vercel login`` (browser OAuth)
3. `npx vercel --prod --yes`
 - └─ auto-detects Next.js
 - └─ creates project "adaptive-antivirus"
 - └─ builds on Vercel infra (37s)
 - └─ deploys to `adaptive-antivirus.vercel.app` (53s total)
4. Verify all 6 routes return 200 from edge

Phase 9 — Email signup (Resend)

The CTA form was originally presentational — clicking submit just flipped local state. This phase wired it to a real backend.

1. User signs up at `resend.com`, creates API key, pastes it
2. `npm install resend`
3. Create `.env.local` with `RESEND_API_KEY` (gitignored)
4. Build `app/api/signup/route.ts` (POST handler with validation, honeypot, Resend call, structured errors)
5. Build `lib/welcomeEmail.ts` (HTML + text + subject)
6. Update `components/sections/CTA.tsx` to do real fetch with loading/success/error states
7. Test locally with `curl` → 200 ok
8. Push env vars to Vercel: `vercel env add RESEND_API_KEY production`
9. Redeploy: `npx vercel --prod`

Phase 10 — Email refinement (anti-spam)

The first email looked too "marketing" — gradients, decorative numbered circles, eyebrow pill, fake `.example` reply-to address. Rewritten same session.

Removed:

- Gradient backgrounds
- Numbered circle badges
- Eyebrow pill
- Centered alignment
- Fake `.example` reply-to

Added:

- Plain-text-style HTML
- Hidden preheader text
- List-Unsubscribe headers
- `color-scheme: dark light`
- Tight ~80-word body

Part III — Decisions

Why hand-rolled UI primitives, not shadcn CLI?

Two reasons:

1. **Token control.** All primitives reference the same `@theme` CSS variables — adding shadcn would create a parallel set of components keyed off its own theme conventions, leading to drift.
2. **Component count.** The site needed nine specific primitives. Pulling in shadcn's CLI scaffolding for that handful added more friction than writing them.

Trade-off: no automatic updates from the shadcn ecosystem if patterns improve. Acceptable for a thesis project.

Why Tailwind v4 beta over stable v3?

V4's `@theme` block is the cleanest way to declare CSS-variable tokens — much better than v3's `tailwind.config.ts` `theme.extend` spaghetti. Worth the beta risk for a single-developer codebase.

Trade-off: must pin `4.0.0-beta.8` exactly. Bump to stable when v4 GA ships.

Why dark-only theme?

The brand is intentionally dark. A cybersecurity product page in light mode reads as generic SaaS. `next-themes` is configured with `forcedTheme="dark"` to lock it.

Trade-off: ~5% of users prefer light. Accepted.

Why no global state library?

The site is mostly static. The few interactive pieces are self-contained. Adding Zustand/Redux/Context would be infrastructure for problems that don't exist.

Rule of thumb applied: don't add a dependency until two unrelated components need to share state.

Why Resend over alternatives?

OPTION	CONSIDERED?	REJECTED BECAUSE
Resend	✅ chosen	Best free tier (3K/mo), best DX, modern API
Postmark	considered	Better deliverability but no free tier
SendGrid	considered	Heavier API, marketing-bloated docs
AWS SES	considered	Setup overhead disproportionate to thesis scope
Gmail SMTP via Nodemailer	considered	Less professional from-address; daily cap

Why per-route unique heroes (V2)?

V1 had every route opening with the same `Hero` component. Visually, every page felt identical above the fold. V2 added four route-specific heroes (PlatformHero, HowHero, CoverageHero, PricingHero) plus a route-unique section per page (EngineSpec, CycleDiagram, OSWall, PricingFAQ). The demo route was added with the same pattern (DemoHero + InteractiveDemo).

The result: each route has a distinct visual identity above the fold while reusing every shared component below it.

Why static pre-rendering for everything?

Every non-API route is static-renderable: no per-user data, no auth, no dynamic content. Static gives us:

- Instant page loads from CDN edge
- Zero serverless function cost for page views
- Trivial scaling

The only function call is `/api/signup`, which runs on demand.

Why the fake threat name `TROJAN.SIM.0XF7A2` ?

In the demo, the simulated payload needs a realistic-looking name (uppercase, hex-y) but must not be mistaken for real malware in screenshots, screen recordings, or content scanners. `.SIM.` in the middle is a clear visual flag that this is a simulation.

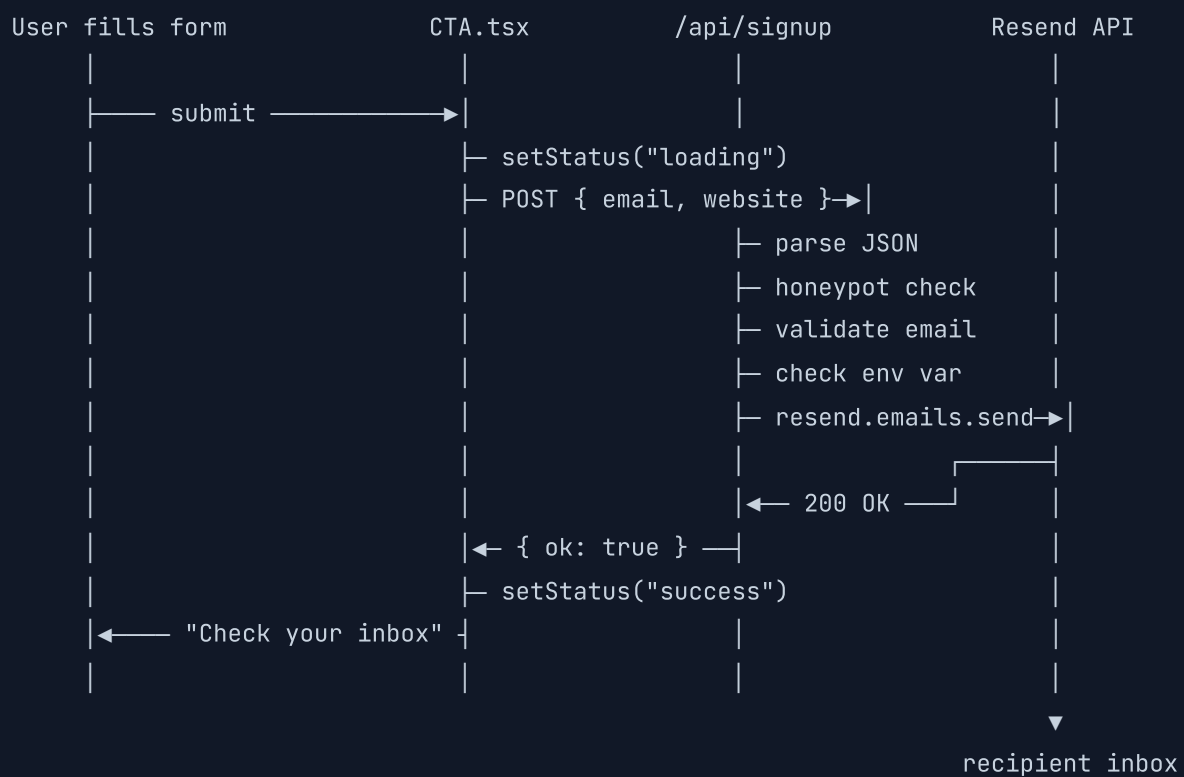
Part IV — Reference

File map

adaptive-antivirus/	
├─ app/	
│ ├─ layout.tsx	Root layout, fonts, JSON-LD, Navbar, ThemeProvider
│ ├─ page.tsx	Home – composes 20 sections in order
│ ├─ globals.css	Design tokens (@theme), utilities, keyframes
│ ├─ platform/page.tsx	PlatformHero + EngineSpec + Architecture + ...
│ ├─ how-it-works/page.tsx	HowHero + CycleDiagram + HowItWorks + ...
│ ├─ coverage/page.tsx	CoverageHero + OSWall + Coverage + ...
│ ├─ demo/page.tsx	DemoHero + InteractiveDemo + DemoExplainer
│ ├─ pricing/page.tsx	PricingHero + Pricing + Competitive + PricingFAQ
│ └─ api/	
│ └─ signup/route.ts	POST handler – Resend integration
├─ components/	
│ ├─ ui/	Atomic primitives (9 files)
│ ├─ shared/	Cross-section components (8 files)
│ └─ sections/	
│ ├─ heroes/	Route-specific heroes (5 files)
│ └─ *.tsx	Home + route-unique sections (~24 files)
├─ hooks/	
│ ├─ useInViewport.ts	IntersectionObserver wrapper
│ └─ useCountUp.ts	Easing count-up (built but not yet wired)
├─ lib/	
│ ├─ utils.ts	cn() – clsx + tailwind-merge
│ ├─ motion.ts	Framer Motion variants + easings
│ ├─ constants.ts	ALL section copy data (single source of truth)
│ └─ welcomeEmail.ts	Welcome email subject + HTML + text
├─ public/	
│ ├─ favicon.svg	Inline SVG shield + cyan gradient
│ └─ og-image.svg	1200×630 OpenGraph image
├─ .env.example	Documents required env vars
└─ .env.local	Local secrets (gitignored)

- └─ .gitignore
- └─ CLAUDE.md Handoff context for future sessions
- └─ ARCHITECTURE.md This file
- └─ README.md User-facing setup & overview
- └─ next.config.ts
- └─ package.json
- └─ postcss.config.mjs
- └─ tsconfig.json

Request flow — signup form



Page composition pattern

Every page follows the same shape:

```

export default function PageName() {
  return (
    <
      <PageHero />           {/* unique above-the-fold */}
      <UniqueSection />      {/* one section unique to this route */}
      <SharedSectionA />     {/* reused from home */}
      <SharedSectionB />     {/* reused from home */}
      <CTA />                {/* always second-to-last */}
      <Footer />             {/* always last */}
    </
  );
}

```

This keeps page files thin (typically 15–25 lines) and makes the routing logic obvious from a file diff.

Dependency graph (high-level)

```

Pages
├── Hero / route hero
├── Sections (20+)
│   ├── Shared components
│   │   └── UI primitives
│   └── UI primitives
└── CTA → /api/signup → lib/welcomeEmail.ts → Resend SDK

```

How to verify the system end-to-end

```

# Local
npm install
npm run typecheck           # exits 0
npm run build                # 9 routes prerender
npm run dev                  # localhost:3000
curl localhost:3000/         # 200
curl -X POST -H 'Content-Type: application/json' \
  -d '{"email":"akshajkarimanikkam@gmail.com"}' \
  localhost:3000/api/signup  # {"ok":true}

# Production
curl https://adaptive-antivirus.vercel.app/  # 200

```


If the signup curl returns 200 but no email arrives, the issue is the Resend free-tier restriction (only the account's signup address can receive). See [CLAUDE.md](#) Open TODOs.